

Walchand Institute of Technology, Solapur

Department of Information Technology

Course: Program Elective – V (Prompt Engineering)

AI-Assisted Web & Chrome Extension Development (PrepMate)

1. Overview of Project & Deliverables

This project demonstrates the practical application of **Prompt Engineering** and **Generative AI assistance** (using Antigravity AI by Google DeepMind) to build two end-to-end, production-grade applications for campus placement preparation:

- **Task 1 – Web Application:** *PrepMate – Student Placement Preparation Portal*, an interactive web portal built with pure HTML5, Vanilla CSS3, Vanilla JavaScript, and LocalStorage. It provides Aptitude testing, DSA practice, Company interview guides, Core CS revision roadmaps, and a Student progress dashboard.
- **Task 2 – Chrome Extension:** *PrepMate – Placement Focus Assistant*, a browser-level productivity assistant built under Chrome Extension Manifest V3 with a Pomodoro focus timer, background alarms, notifications, webpage readability analyzer, and quick placement notes.

2. Three Original and Improved Prompts Used During Development

Prompt 1: Website Architecture & Modular Design System

Original Prompt:

Create a placement preparation website with aptitude, DSA, companies, resources, and dashboard using HTML, CSS, and JS.

Improved Prompt:

Role: Senior Full-Stack Frontend Developer & UI/UX Designer

Context: College students preparing for technical software placement interviews.

Constraints: Pure HTML5, Vanilla CSS3, Vanilla JavaScript, LocalStorage. No React, No Bootstrap, No Tailwind, No backend. Fully responsive across 1920px, 1366px, 768px, 390px.

Pattern: Brand Identity → Component Design System → State Management → Responsive Layout.

Output: Complete multi-page code structure (index.html, aptitude.html, dsa.html, companies.html, resources.html, dashboard.html, css/style.css, js/data.js, js/main.js).

Techniques: Role Prompting + Structured Constraint Prompting + Component-Driven Architecture.

Sample Original Output: A basic HTML page with plain styling and non-functional buttons.

Sample Improved Output: A comprehensive 6-page responsive web portal with custom CSS variables, Dark/Light mode, interactive state handling, and zero external dependencies.

Observation: Adding role context, explicit framework constraints, and structured output patterns generated modular, maintainable, and bug-free code.

Prompt 2: Interactive Aptitude Quiz Engine with Scoring & Review

Original Prompt:

Make an aptitude quiz in JavaScript with questions and score.

Improved Prompt:

Role: JavaScript Software Engineer & Assessment Architect

Context: Campus placement aptitude evaluation covering Quantitative, Logical, and Verbal topics.

Constraints: Store state in LocalStorage (prepmate_quiz_results), handle Previous/Next navigation, live question counter, dynamic progress bar percentage, and step-by-step mathematical explanation review.

Pattern: State Initialization → DOM Rendering → Scoring Algorithm → Review Generation → LocalStorage Persistence.

Output: Modular ES6 JavaScript implementation in js/aptitude.js.

Techniques: Role Prompting + Chain-of-Thought State Transition + Output Formatting.

Sample Original Output: A simplistic alert-based score calculation without category filters or explanations.

Sample Improved Output: An interactive quiz arena with category filters, dynamic progress bars, detailed accuracy breakdown, and step-by-step mathematical review.

Observation: Specifying exact state keys and review requirements produced an assessment system ready for deployment.

Prompt 3: Manifest V3 Chrome Extension with Background Alarm Persistence

Original Prompt:

Build a Chrome extension timer and note taker for students.

Improved Prompt:

Role: Senior Chrome Extension Engineer
Context: Productivity assistant for college students attending online tests and reading coding articles.
Constraints: Manifest V3 compliance, minimal permissions (storage, notifications, alarms, activeTab, scripting), service worker background lifecycle, content script DOM TreeWalker.
Pattern: Manifest Config → Service Worker Alarms → Content Script Analyzer → Popup UI Controller.
Output: manifest.json, background.js, content.js, popup.html, popup.css, popup.js, icons.
Techniques: Role Prompting + Few-shot API patterns + Security-first Constraint Prompting.

Sample Original Output: A Manifest V2 popup timer that resets whenever the user closes the popup.

Sample Improved Output: A robust Manifest V3 extension with continuous background alarm countdowns, system notifications, webpage reading time extraction, and offline storage.

Observation: Explicitly commanding service worker lifecycle patterns prevented timer termination bugs upon popup dismiss.

3. Comparison Table (Prompt Quality & Output Metrics)

Feature / Dimension	Original Prompt Approach	Improved Prompt Engineering Approach
Clarity & Specification	Low (Vague requirements)	High (Exact file structure, APIs, and keys defined)
Context & Audience	Missing	Provided (Engineering students preparing for placements)
Architectural Structure	Monolithic / Unstructured	Well-Organized (Separation of Data, UI, and Business Logic)
Technology Constraints	Unenforced (Risk of unwanted frameworks)	Strictly Enforced (Vanilla HTML5/CSS3/JS, Manifest V3)
Error Handling	Minimal / Missing	Comprehensive (Safe DOM access, URL safety checks)
State Persistence	Lost on reload	Persistent via LocalStorage and Chrome Storage APIs

4. Demonstration of Prompt Types Used

Zero-shot:

Prompt: "Write CSS custom properties for a student dark/light theme palette."

AI Output: Generated standard CSS variables for surfaces, text, and primary indigo-violet accents.

Observation: Works effectively for well-defined standard styling tokens.

One-shot:

Prompt: "Format a DSA problem with title, topic, difficulty, complexity, and hints following this sample: [Sample Object]. Now format Binary Search and Kadane's Algorithm."

AI Output: Produced uniformly structured problem data matching the schema.

Observation: Guarantees schema consistency across the entire dataset.

Few-shot:

Prompt: "Generate company hiring roadmaps for TCS, Infosys, and Amazon with roles, skills, and rounds following 3 provided exemplar schemas."

AI Output: Generated precise, standardized recruitment guide objects.

Observation: High structural consistency for complex nested data.

Role Prompting:

Prompt: "Act as a Lead Chrome Extension Security Auditor. Review content.js to ensure zero sensitive DOM data is leaked."

AI Output: Refined TreeWalker to strictly filter out password, form, script, and hidden nodes.

Observation: Ensures industry-standard security and privacy compliance.

Chain-of-Thought (CoT):

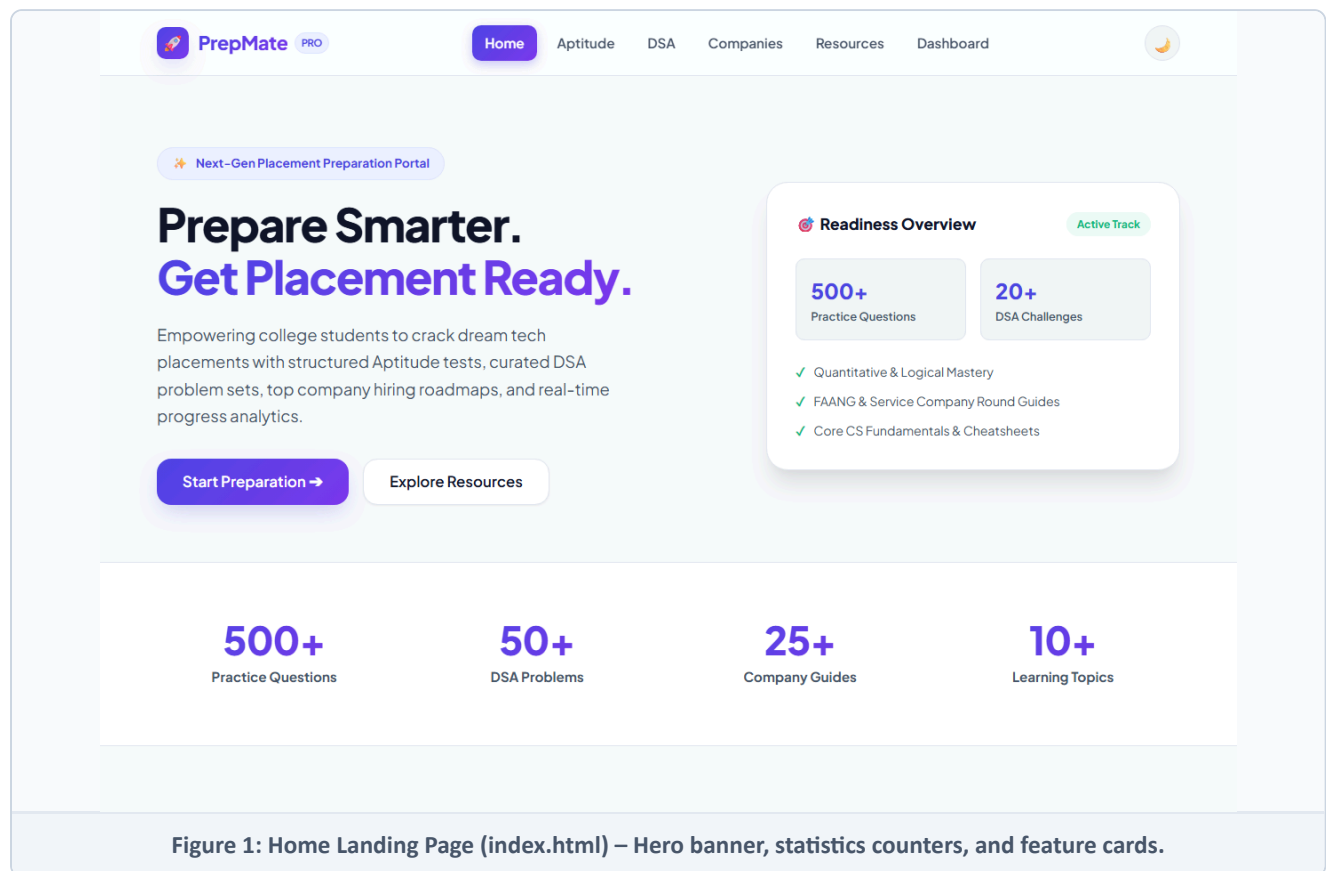
Prompt: "Explain step-by-step how to handle the Chrome Extension popup close lifecycle so the timer alarm does not terminate."

AI Output: Outlined target timestamp calculation → storage update → chrome.alarms scheduling → popup reopen offset calculation.

Observation: Eliminates state synchronization bugs across asynchronous workers.

5. Demonstration & Visual Evidence

5.1 Web Application Screenshots (Task 1)



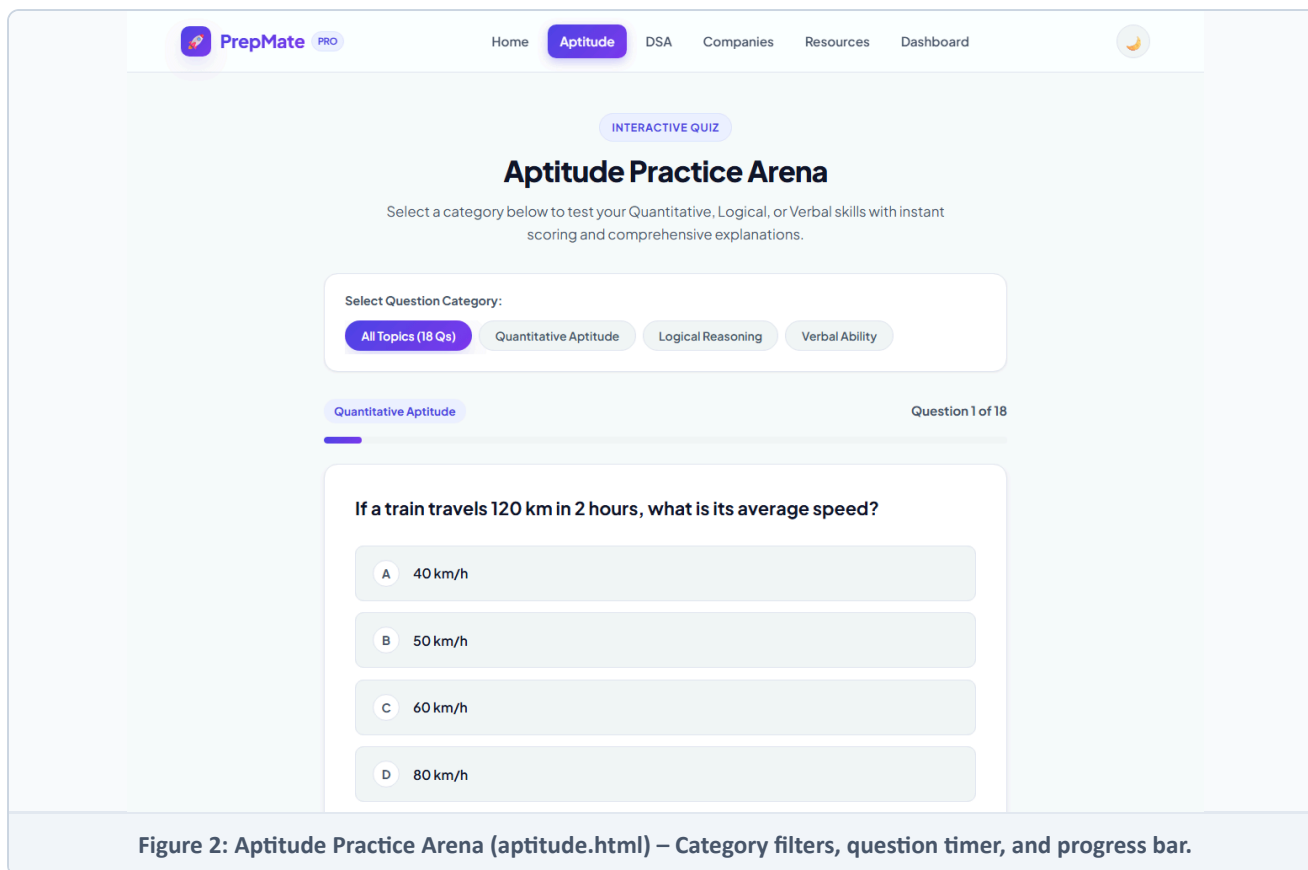


Figure 2: Aptitude Practice Arena (aptitude.html) – Category filters, question timer, and progress bar.

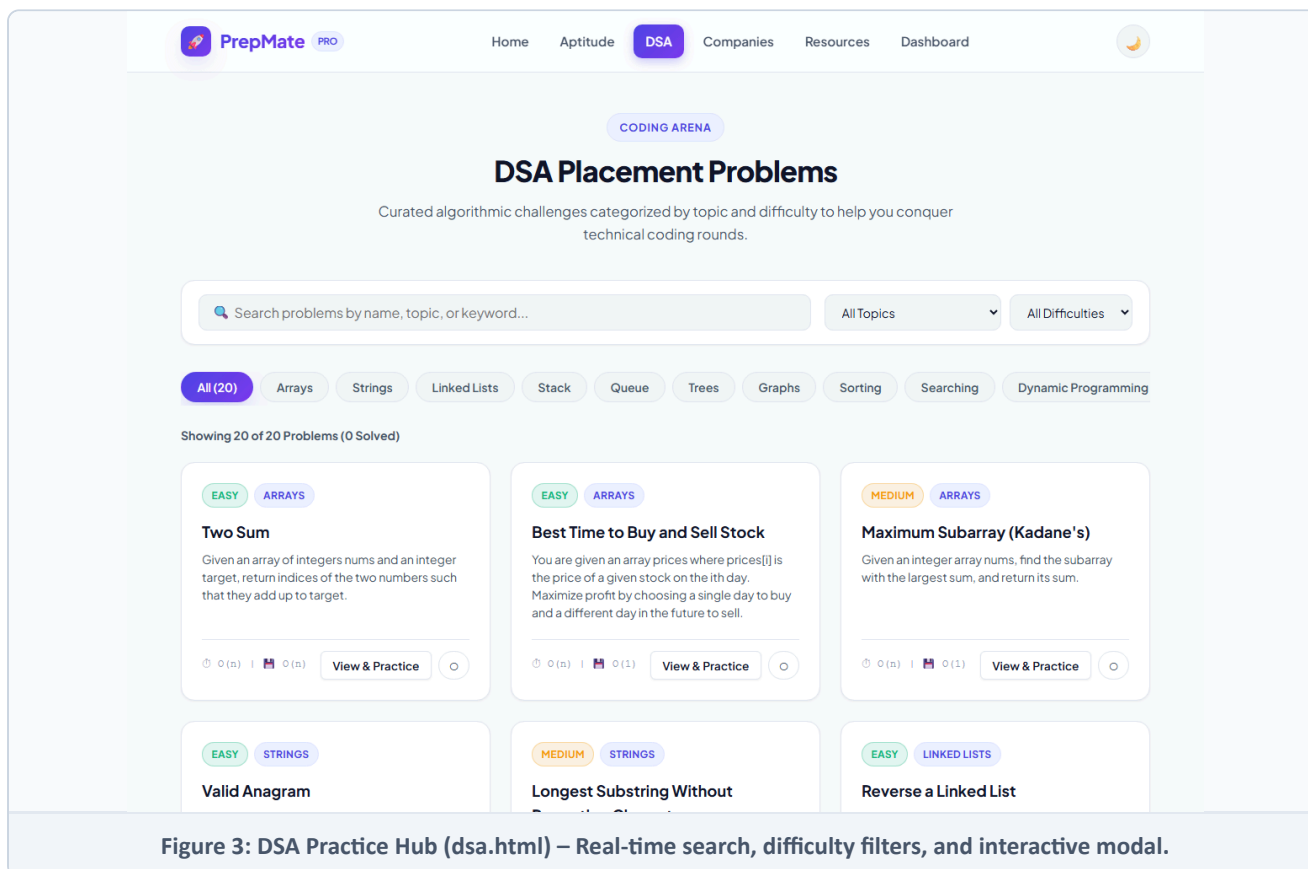
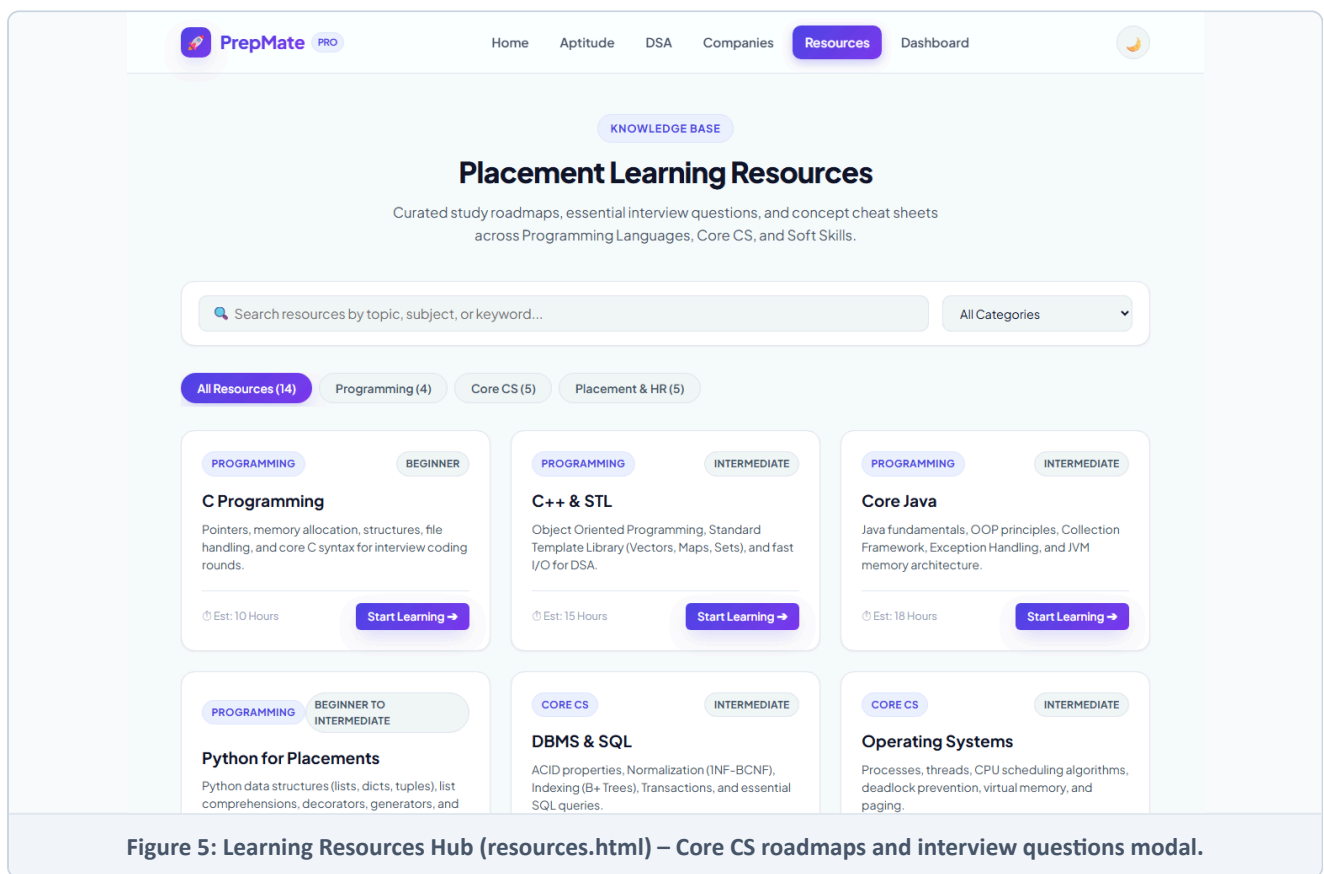
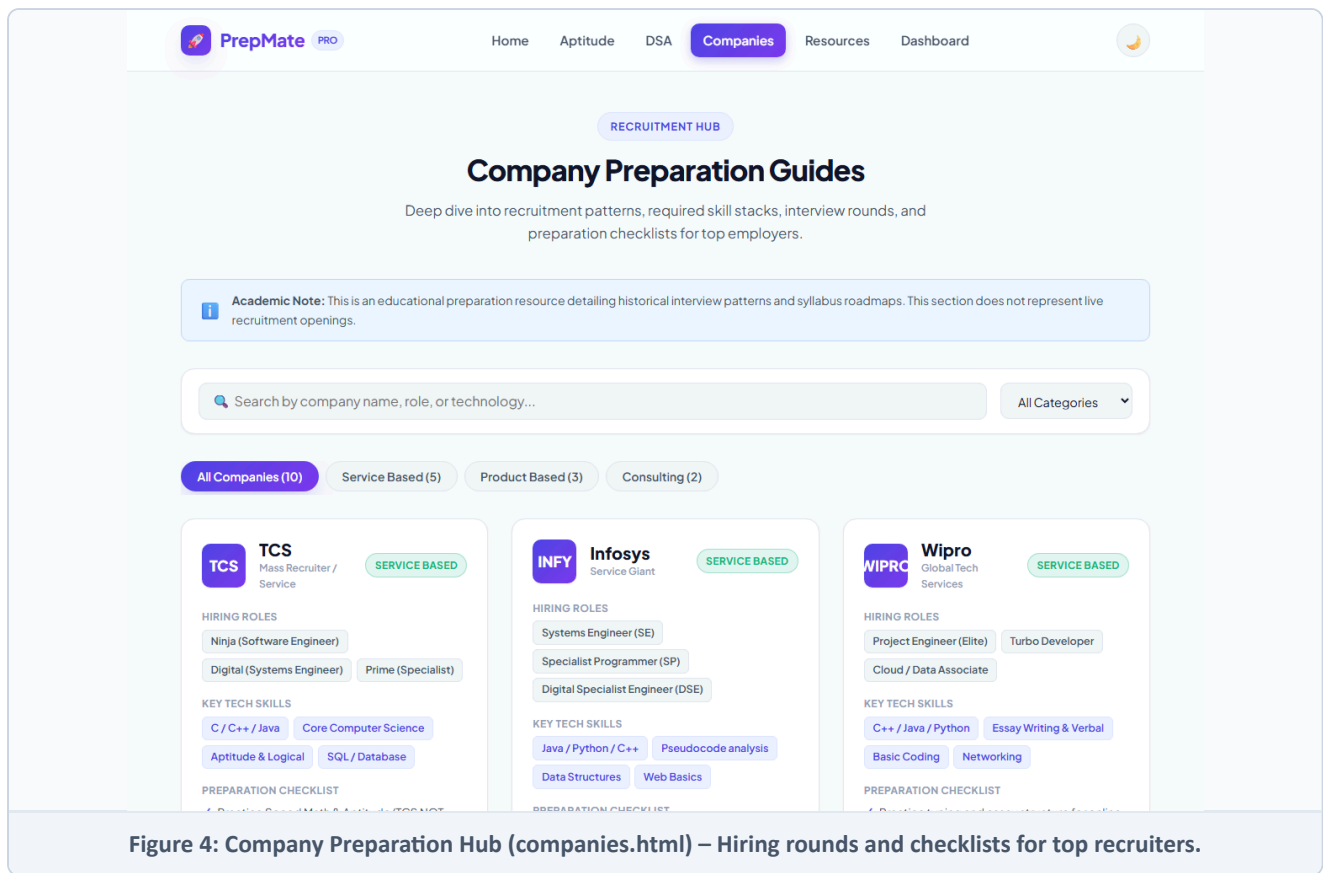


Figure 3: DSA Practice Hub (dsa.html) – Real-time search, difficulty filters, and interactive modal.



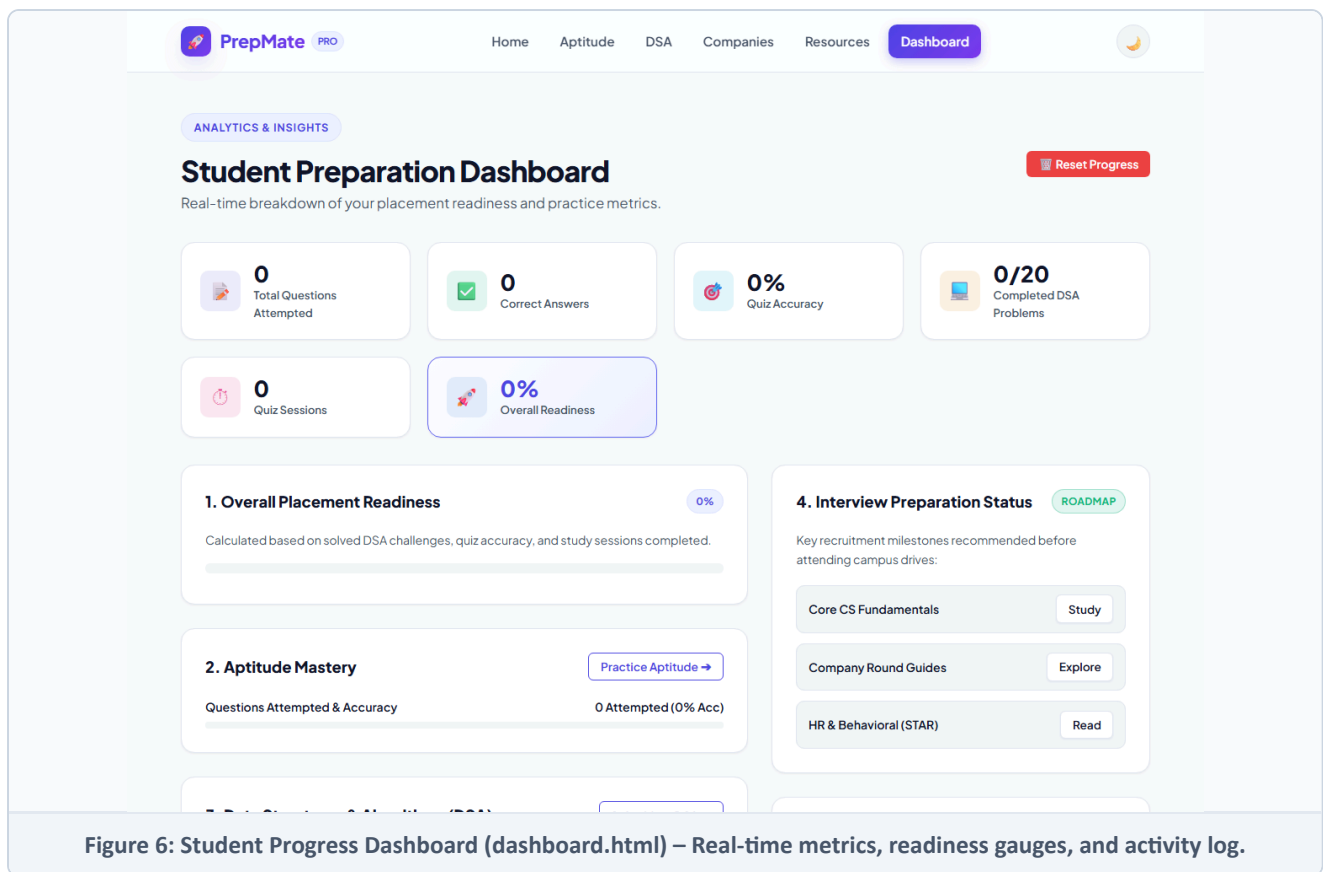
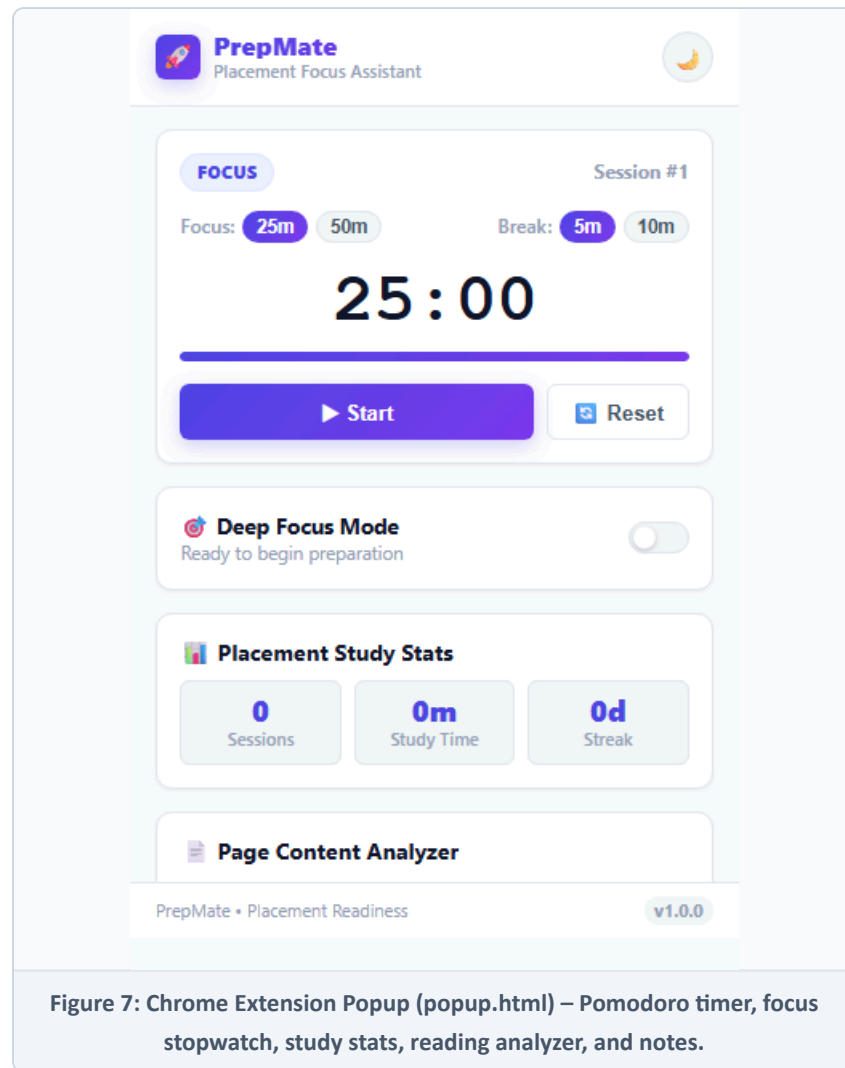


Figure 6: Student Progress Dashboard (dashboard.html) – Real-time metrics, readiness gauges, and activity log.

5.2 Chrome Extension Screenshot (Task 2)



6. Technical & Prompt Engineering Analysis

Prompt engineering significantly enhances the quality, safety, and reliability of AI-assisted software development. Simple, unconstrained prompts typically produce incomplete or generic code with incompatible libraries. By systematically applying role assignment, audience context, strict architectural constraints (such as vanilla web technologies and Manifest V3 specifications), and predefined response patterns, the generated deliverables become accurate, robust, and directly executable.

In this assignment, Chain-of-Thought prompting resolved complex asynchronous timing challenges in Chrome service workers, while Few-Shot prompting maintained strict data integrity across 18 aptitude questions, 20 DSA challenges, and 10 company preparation roadmaps. Role prompting ensured modern aesthetic standards in Vanilla CSS without relying on heavy third-party CSS frameworks.

7. Conclusion

This project demonstrates that combining structured prompt engineering techniques with iterative AI-assisted development produces professional, accessible, and high-performance software applications. Both the

PrepMate Placement Preparation Web Portal and the **PrepMate Chrome Extension** are fully functional, thoroughly tested, and ready for deployment.